

Parametric Generalized Hamiltonian Neural Networks

Philipp Horn^{a,*}, Barry Koren^a

^a*Centre for Analysis, Scientific Computing and Applications, Eindhoven University of Technology, The Netherlands*

Abstract

We present a neural network architecture based on the Generalized Hamiltonian Neural Networks that can learn a parameter-dependent flow map while preserving the symplectic structure. These Parametric Generalized Hamiltonian Neural Networks have great approximation capabilities which is highlighted by a universal approximation theorem. We compare these neural networks against physics-unaware multilayer perceptrons, for which integrating a parameter dependency can be easily achieved by adding an additional input neuron. The Parametric Generalized Hamiltonian Neural Networks are able to extrapolate to areas of the phase space without training data, while the physics-unaware multilayer perceptrons are not. The Parametric Generalized Hamiltonian Neural Networks achieve this with the same prediction speed as multilayer perceptrons and one order of magnitude less trainable parameters.

Keywords: Neural Networks, Hamiltonian Systems, Scientific Machine Learning, Structure Preservation, Symplectic Algorithms

1. Introduction

Many different approaches to structure-preserving neural networks for Hamiltonian systems have been developed [1, 2, 3, 4, 5, 6]. However, none of these neural network architectures have been introduced for parameter-dependent Hamiltonian systems. The parameters of the Hamiltonian systems are always fixed and defined through the data. For every change in a parameter, a new dataset needs to be created, and a new neural network needs to be trained.

Before training a neural network, one must decide on a neural network architecture, such as a multilayer perceptron, convolutional neural network, transformer, etc. Together with the topology hyperparameters, this defines the search space in which the optimization algorithm tries to find a minimum of the loss function during training. This space of functions is also known as the hypothesis space.

A structure-preserving neural network embeds a prior known constraint (or structure) in the neural network's topology itself. For scientific computing applications, these structures are usually physical constraints. In contrast to physics-informed neural networks [7], in which the constraint is added as an extra loss term, the embedded structure in structure-preserving neural networks implies a change in the hypothesis space. With this modified hypothesis space comes the potential for better generalization of the neural networks and the need for considerably fewer trainable parameters. However, since it is a restriction of the search space, it bears the risk of reduced expressivity and a space that is harder for the training algorithm to navigate.

In the following sections, we first introduce the necessary background knowledge on Hamiltonian systems in Section 2 and discuss well-known symplectic neural networks in Section 3. In Section 4, we introduce new Parametric Hamiltonian Neural Networks. They are capable of learning parameter-dependent flow maps of Hamiltonian systems and eliminate the need to train a new neural network for every parameter value. These neural networks are still structure-preserving, and a universal approximation theorem is provided in Section 4.1. Finally, an extensive comparison of different neural network architectures is performed through numerical experiments in Section 6.

*Corresponding author

Email address: p.horn@tue.nl (Philipp Horn)

2. Hamiltonian Systems

Hamiltonian systems are a widely used class of dynamical systems. They occur in classical mechanics and other areas of physics. In the choice of systems we use for testing in Section 6, we limit ourselves to Hamiltonian systems from mechanics. The phase space of a Hamiltonian system is always of even degree and consists of the generalized momenta $\mathbf{p} \in \mathbb{R}^d$ and the generalized positions $\mathbf{q} \in \mathbb{R}^d$. The following first-order ordinary differential equation describes the time evolution of these systems:

$$\begin{pmatrix} \dot{\mathbf{p}} \\ \dot{\mathbf{q}} \end{pmatrix} = -J\nabla H(\mathbf{p}, \mathbf{q}), \quad \text{with } J = \begin{pmatrix} 0 & I_d \\ -I_d & 0 \end{pmatrix}. \quad (1)$$

This equation is better known as Hamilton's equation of motion. The function $H : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ in equation (1) is called the Hamiltonian function (or shortly: the Hamiltonian). In parameter-dependent Hamiltonian systems, the Hamiltonian of the system depends on one or more parameters in addition to the momenta and positions. These parameters can include masses or other properties of the objects in the system. All k parameters are combined in a parameter vector $\boldsymbol{\mu} \in \mathbb{R}^k$, and the Hamiltonian is then a function $H : \mathbb{R}^d \times \mathbb{R}^d \times \mathbb{R}^k \rightarrow \mathbb{R}$.

The Hamiltonian of a system measures the total energy of the system and is constant along the trajectories of that Hamiltonian system. In a subclass of Hamiltonian systems, the Hamiltonian is separable. This means it can be split into a sum of one part that only depends on the momenta $T(\mathbf{p})$ and a second part that only depends on the positions $U(\mathbf{q})$. In mechanics, one refers to the first as kinetic energy and the second as potential energy. In parameter-dependent Hamiltonian systems, both kinetic and potential energy can depend on the parameters $\boldsymbol{\mu}$.

While total energy conservation is a key property of Hamiltonian systems from a physics point of view, the symplectic structure of Hamiltonian systems is even more important for scientific computing. The symplectic structure refers to the flow map $\varphi_h : \mathbf{x}(t) \mapsto \mathbf{x}(t+h)$ of a Hamiltonian system obeying the following equation:

$$\left(\frac{\partial \varphi_h}{\partial \mathbf{x}} \right)^T J \frac{\partial \varphi_h}{\partial \mathbf{x}} = J, \quad \mathbf{x} = \begin{pmatrix} \mathbf{p} \\ \mathbf{q} \end{pmatrix} \in \mathbb{R}^{2d}. \quad (2)$$

A numerical integrator used to solve a Hamiltonian system approximates the flow map of the system. It is well established that it is highly beneficial to preserve this symplectic structure in the numerical integrator [8]. These so-called symplectic integrators possess desirable properties, such as long-term energy conservation and stability for many Hamiltonian systems.

3. Symplectic Neural Networks

Since preserving the symplectic structure in a numerical integrator proves highly beneficial, there are multiple neural network architectures that replicate this. A neural network can be used in the same iterative process as a numerical integrator. To predict the trajectories of a Hamiltonian system, the neural network is trained on data from many trajectories with different initial conditions and with a constant step size h between the recorded states. The feature-label pairs of the training data consist of the recorded states and the corresponding states a step size h later along the same trajectory:

$$S = \bigcup_{1, \dots, M} \{(\mathbf{x}_n, \varphi_h(\mathbf{x}_n) = \mathbf{x}_{n+1}) | n = 0, \dots, N-1\}. \quad (3)$$

This means that the neural network also approximates the flow map of the Hamiltonian system.

3.1. SympNets and HénonNets

Two successful neural network architectures that preserve the symplectic structure of the flow map and fulfill a universal approximation theorem are the SympNets [3] and the HénonNets [4]. Both focus on learning rather basic symplectic maps, and through a concatenation of these symplectic maps, they are able to approximate complex flow maps of Hamiltonian systems. A SympNet is built by concatenating many so-called unit triangular updates

$$f_{\text{up}} \begin{pmatrix} \mathbf{p} \\ \mathbf{q} \end{pmatrix} := \begin{pmatrix} \mathbf{p} + \nabla V(\mathbf{q}) \\ \mathbf{q} \end{pmatrix}, \quad f_{\text{low}} \begin{pmatrix} \mathbf{p} \\ \mathbf{q} \end{pmatrix} := \begin{pmatrix} \mathbf{p} \\ \mathbf{q} + \nabla V(\mathbf{p}) \end{pmatrix}, \quad (4)$$

and the trainable parameters of each layer are in the gradient functions ∇V of these updates. On the other hand, a HénonNet consists of Hénon maps

$$h \begin{pmatrix} p \\ q \end{pmatrix} := \begin{pmatrix} -q + \nabla V(p) \\ p + \eta \end{pmatrix}, \quad (5)$$

that are repeated four times per layer. The trainable parameters are again in the gradient functions ∇V and additionally in the shift vectors η . Both SympNets and HénonNets preserve the overall symplectic structure of the flow map since concatenations of symplectic maps are themselves symplectic maps.

While these neural networks are universal approximators of the flow maps of Hamiltonian systems, they cannot be trained on data with varying parameters of the Hamiltonian system. For every parameter value, a new neural network needs to be trained. Furthermore, it is not obvious how a parameter dependency can be included in the neural network architecture without destroying the symplectic structure.

3.2. Generalized Hamiltonian Neural Networks

More recently, a generalized framework called Generalized Hamiltonian Neural Networks (GHNNs) that unites multiple symplectic neural networks (including SympNets and HénonNets) was presented [6]. It is possible to introduce this general framework by taking a new point of view on these neural networks. The unit triangular updates and Hénon layers are equivalent to the integration steps of symplectic integrators with learned Hamiltonians.

Therefore, a Generalized Hamiltonian Neural Network is a concatenation of symplectic integrators. Each integrator includes a separable Hamiltonian. The kinetic and potential energies of these Hamiltonians are parametrized as multilayer perceptrons. A visualization of these neural networks is given in Figure 1.

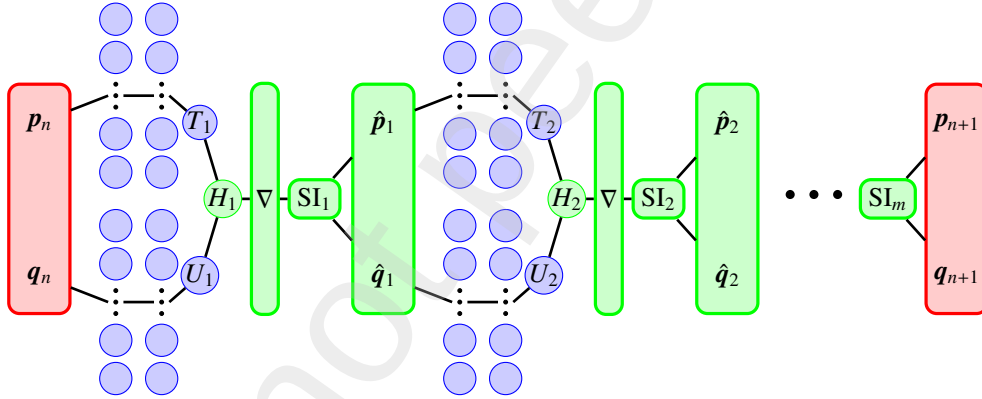


Figure 1: Exemplary schematic of a Generalized Hamiltonian Neural Network with two hidden layers per kinetic and potential energy.

For the Generalized Hamiltonian Neural Networks, the same universal approximation theorem as for the SympNets and HénonNets holds. Also, just like the SympNets and HénonNets, a new Generalized Hamiltonian Neural Network needs to be trained for each parameter value in the Hamiltonian system. However, the Generalized Hamiltonian Neural Networks present a natural way to build a parameter dependency into the neural network architecture.

4. Parametric Generalized Hamiltonian Neural Networks

To build a parameter dependent version of the Generalized Hamiltonian Neural Networks, we take a closer look at them. The output of a Generalized Hamiltonian Neural Network is calculated by a concatenation of symplectic integrators (SI):

$$\text{GHNN}(p, q) = \text{SI}_m(H_m, \cdot, \cdot) \circ \dots \circ \text{SI}_1(H_1, p, q). \quad (6)$$

Each learned Hamiltonian consists of two multilayer perceptrons: one that receives as input p and calculates the kinetic energy, and one that receives q as input and calculates the potential energy.

In a parameter-dependent Hamiltonian system, the parameter enters Hamilton's equation of motion (1) through the Hamiltonian H being dependent on the parameters. Therefore, a natural way to include a dependency on the

parameter μ in the Generalized Hamiltonian Neural Network is to use this parameter as an additional input for the multilayer perceptrons, calculating the potential and kinetic energies in each integrator:

$$\text{PGHNN}(\mathbf{p}, \mathbf{q}, \mu) = \text{SI}_m(H_m, \cdot, \cdot) \circ \dots \circ \text{SI}_1(H_1, \mathbf{p}, \mathbf{q}), \quad (7)$$

$$H_i(\mathbf{p}, \mathbf{q}, \mu) = T_i(\mathbf{p}, \mu) + U_i(\mathbf{q}, \mu). \quad (8)$$

With the outputs of the multilayer perceptrons now also depending on the parameter μ , it is important to ensure that the gradients in the integrators are only taken with respect to the momenta and positions. Furthermore, the parameter μ needs to be passed unchanged from integrator to integrator so that it can be used as input in every multilayer perceptron. A schematic of these Parametric Generalized Hamiltonian Neural Networks (PGHNNs) is given in Figure 2.

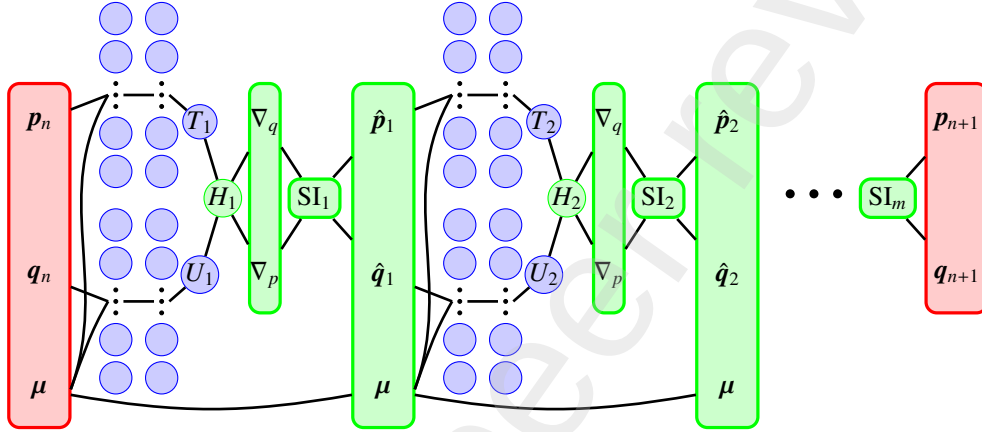


Figure 2: Exemplary schematic of a Parametric Generalized Hamiltonian Neural Network with two hidden layers per kinetic and potential energy.

All neural networks discussed so far require a fixed step size h in the training data. If one wants to use a neural network to learn a flow map with a different step size, a new neural network needs to be trained. Furthermore, if the data is measured from a real-world Hamiltonian system, there might be a variation in the step size. In the paper introducing the SympNets [3], a possible solution to this limitation was proposed. By multiplying the gradient function ∇V in the unit triangular updates (Equation (4)) with the step size h , a dependency on the step size can be achieved in the predictions of the neural networks.

In the new point of view on the SympNets introduced by the Generalized Hamiltonian Neural Networks, this multiplication by h simply translates to an adjustment of the step size in the symplectic integrators. This is a rather natural way to include a dependency on the step size in the neural network, although it is limited to a linear dependency only. The updates of each layer are simply scaled by h . A more general and complex dependency on the step size h can be achieved through the Parametric Generalized Hamiltonian Neural Networks by including the step size as an additional parameter in the parameter input μ . We compare the different possibilities to include a varying step size through a numerical experiment in Section 6.3.

Table 1 offers an overview of the benefits and drawbacks of different neural network architectures that can be trained on data from Hamiltonian systems. Hamiltonian neural networks (HNNs) [1] and Symplectic Recurrent Neural Networks (SRNNs) [2] were both introduced without specifically mentioning their ability to be used in parametric Hamiltonian systems. For both, a parameter dependency can be easily added by introducing an additional input neuron for the learned Hamiltonian. Since both HNNs and SRNNs suffer from additional drawbacks, we do not compare them against the PGHNNs in the numerical experiment in Section 6.3.

4.1. Approximation Capabilities of PGHNNs

For the SympNets, a universal approximation theorem was proven. It shows that every r -smooth symplectic map can be approximated uniformly in the C^r -norm on compacta with arbitrary precision by a SympNet. The same universal approximation theorem holds for the HénonNets, and since the GHNNs are a more general class of functions,

Table 1: Comparison of the capabilities of the different neural network architectures that were specifically introduced for data of Hamiltonian systems and multilayer perceptrons.

	MLP	HNN	SRNN	SympNet	HénonNet	PGHNN
No time derivative data	✓	✗	✓	✓	✓	✓
Strictly symplectic	✗	✗	✓	✓	✓	✓
Non-separable Hamilt.	✓	✓	✗	✓	✓	✓
Parameter-dependent	✓	✓	✓	✗	✗	✓
Varying step sizes	✓	✓	✓	✓	✗	✓

they are also universal approximators of symplectic maps. In this section, we aim to extend this universal approximation theorem to parameter-dependent symplectic maps and show that Parametric Generalized Hamiltonian Neural Networks can approximate these maps arbitrarily closely. To do this, let us first start with a few definitions.

The first is a common short-hand notation for derivatives of multivariate functions:

Definition 1. Short notation for the multivariate derivative of the function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is given by:

$$D^\alpha f = \frac{\partial^{|\alpha|} f}{\partial x_1^{\alpha_1} \cdots \partial x_n^{\alpha_n}}, \quad \alpha \in \underbrace{\mathbb{N}^0 \times \cdots \times \mathbb{N}^0}_{n\text{-times}}, \quad |\alpha| = \alpha_1 + \cdots + \alpha_n. \quad (9)$$

With this notation, we can define the C^r -norm on compacta as:

Definition 2. For any $r \in \mathbb{N}^0$, $W \subset \mathbb{R}^n$ compact, the norm of a function $f \in C^r(W; \mathbb{R}^m)$ is defined as:

$$\|f\|_{C^r(W; \mathbb{R}^d)} := \sum_{|\alpha| \leq r} \max_{1 \leq i \leq m} \sup_{x \in W} |D^\alpha f_i(x)|. \quad (10)$$

Let us now define the class of functions we want to approximate with PGHNNs:

Definition 3 (C^r -smooth parameter-dependent symplectic maps). For any $r \in \mathbb{N}^+$ and open $U \subset \mathbb{R}^{2d} \times \mathbb{R}^k$, the set of all C^r -smooth symplectic maps is defined as:

$$\mathcal{SP}^r(U) = \left\{ \varphi \in C^r(U; \mathbb{R}^{2d}) \left| \left(\frac{\partial \varphi}{\partial \mathbf{x}} \right)^T J \frac{\partial \varphi}{\partial \mathbf{x}} = J \right. \right\}, \quad (11)$$

for which $\frac{\partial \varphi}{\partial \mathbf{x}}$ denotes the Jacobian matrix of φ with respect to the first $2d$ arguments.

Before we come to the universal approximation theorem of PGHNNs, we will now introduce the concept of r -finite activation functions:

Definition 4 (r -finite). For $r \in \mathbb{N}^0$, σ is called r -finite if: $\sigma \in C^r(\mathbb{R})$ and $0 < \int |D^r(\sigma)| d\lambda < \infty$.

Here λ is the Lebesgue measure on $\mathbb{R}$.

With these four definitions, we can now state the universal approximation theorem for PGHNNs. It extends Theorems 4 and 5 in [3] to parameter-dependent symplectic maps and uses PGHNNs instead of SympNets.

Theorem 1 (Universal approximation of PGHNNs). For any $r \in \mathbb{N}^+$, open set $U \subset \mathbb{R}^{2d} \times \mathbb{R}^k$, compact set $W \subset U$, any $\varphi \in \mathcal{SP}^{r+1}(U)$, and any $\epsilon > 0$, there exists a PGHNN φ_{NN} such that

$$\|\varphi - \varphi_{\text{NN}}\|_{C^r(W; \mathbb{R}^{2d})} < \epsilon \quad (12)$$

if the activation functions inside the MLPs learning the Hamiltonians are $(r+1)$ -finite.

The proof of this theorem can be found in Appendix A.

5. Implementation

Similar to the Generalized Hamiltonian Neural Networks, a performance increase can be gained by calculating the gradients of the internal multilayer perceptrons by hand instead of using automatic differentiation. To do this, we have to split the weight matrix in the first hidden layer into one part connected to the coordinate ($\mathbf{p}$ or $\mathbf{q}$) inputs, $W_1 \in \mathbb{R}^{o \times d}$, and one part connected to the parameter inputs, $W_\mu \in \mathbb{R}^{o \times k}$, where o is the number of neurons in the first hidden layer.

With this split, the gradient of a multilayer perceptron with one hidden layer that calculates the kinetic energy is given by:

$$W_1^T \text{diag}(\mathbf{w}) \sigma \left((W_1 \quad W_\mu) \begin{pmatrix} \mathbf{p} \\ \boldsymbol{\mu} \end{pmatrix} + \mathbf{b} \right). \quad (13)$$

Here $\mathbf{b} \in \mathbb{R}^o$ is the bias vector in the one hidden layer, and $\text{diag}(\mathbf{w}) \in \mathbb{R}^{o \times o}$ is a diagonal matrix with the elements of the output weight vector on its diagonal. σ is the derivative of the activation function in the hidden layer of the multilayer perceptron.

Similarly to a multilayer perceptron with one hidden layer, the gradient of a multilayer perceptron with two hidden layers for the kinetic energy can be calculated:

$$W_1^T \text{diag} \left(W_2^T \text{diag}(\mathbf{w}) \sigma \left(W_2 \Sigma \left((W_1 \quad W_\mu) \begin{pmatrix} \mathbf{p} \\ \boldsymbol{\mu} \end{pmatrix} + \mathbf{b}_1 \right) + \mathbf{b}_2 \right) \right) \sigma \left((W_1 \quad W_\mu) \begin{pmatrix} \mathbf{p} \\ \boldsymbol{\mu} \end{pmatrix} + \mathbf{b}_1 \right). \quad (14)$$

All the parameters in this equation are the same as before. Additionally, the second layer's weight matrix W_2 and its bias vector $\mathbf{b}_2$ are used. Also, Σ is the activation function used in the multilayer perceptron.

We can continue to calculate the gradients for more than two layers. However, the expressions become rather lengthy. Furthermore, only up to two hidden layers are used to generate the results of this work. For easy reproducibility, our PyTorch [9] implementation of the PGHNNs and all code used to generate the results (including the data) in the following section can be found on GitHub (<https://github.com/AELITTEN/PGHNN>). The trained neural networks are also available in a GitHub repository (https://github.com/AELITTEN/NeuralNets_PGHNN).

6. Numerical Experiments

Compared to symplectic neural networks, it is straightforward to include a parameter dependency in a physics-unaware multilayer perceptron (MLP) that is trained to approximate the flow map of a Hamiltonian system. In a multilayer perceptron, only one additional input neuron per parameter is needed. No other modifications need to be made.

To test the performance of the Parametric Generalized Hamiltonian Neural Networks and to compare them against multilayer perceptrons, we train them on data from a pendulum and a double pendulum with varying parameters. For both test cases, three types of neural networks are trained. First, a multilayer perceptron with 5 hidden layers; second, a PGHNN with one hidden layer in each of the 10 learned Hamiltonians; and last, a PGHNN with two hidden layers in each of the 5 learned Hamiltonians. The number of neurons is chosen such that the prediction speeds of all neural network topologies are similar. Furthermore, the symplectic Euler method is used as an integrator in the PGHNNs. A list of the hyperparameters and the resulting number of trainable parameters can be found in Table 2.

To validate that PGHNNs are also able to deal with data that has irregular step sizes, we use data from a double pendulum with fixed parameters. With this data, a multilayer perceptron and a SympNet are also trained. The hyperparameters of these neural networks are again given in Table 2. For the SympNets, if the original denotation of [3] is used, 10 learned Hamiltonians with 50 neurons translate to 20 gradient modules with a width of 50.

For every architecture, 10 neural networks are trained with the same topology but with differently initialized weights. This is done to ensure that we do not compare a neural network that became stuck in a poor local minimum with one that found a good local minimum during training. Additionally, it can provide insight into how challenging the loss landscape is to navigate for the minimization algorithm. Overall, the data is split into 80% training data, 10% validation data, and 10% test data for all experiments. Furthermore, the Adam algorithm with the default hyperparameters is used to train all neural networks [10].

Table 2: Hyperparameters of the neural network topologies for the numerical experiments. The MLP does not have learned Hamiltonians. The PGHNNs and SympNets on the other hand include two MLPs per learned Hamiltonian. The hidden layers in the PGHNNs and SympNets are the number of hidden layers in these MLPs. For the SympNets, 10 learned Hamiltonians with 50 neurons translate to 20 gradient modules with a width of 50.

NN architecture	Learned Hamilt.	Hidden layers	Neurons per layer	Trainable parameters		
				Pendulum	Double pendulum	Doub. pend. (varying h)
MLP	-	5	128	66818	67460	67332
PGHNN	10	1	50	4000	6000	5000
PGHNN	5	2	25	7500	8000	7750
SympNet	10	1	50	-	-	4000

6.1. Pendulum

The single pendulum is a two-dimensional ($2d = 2$) Hamiltonian system built out of a rod with a mass at its end that can swing freely in two dimensions. The parameter in this Hamiltonian system that can change is the mass m , the length of the rod l , or the gravitational acceleration g . For simplicity, we choose dimensionless values for these parameters and fix the mass and the gravitational acceleration ($m = 1, g = 1$). The resulting Hamiltonian of this system is given by:

$$H(p, q, l) = \frac{p^2}{2l^2} + l(1 - \cos(q)). \quad (15)$$

The length of the rod is the parameter that is varied and therefore needs to be the parameter input μ to the different neural networks. The dataset consists of 500 trajectories, for which the initial angle q is drawn uniformly at random between $[-0.95\pi, 0.95\pi]$, and the initial momentum p is zero. We let the pendulum drop from its initial angle. The length of the pendulum is also drawn uniformly from $[0.5, 1.5]$. The trajectories are calculated using the symplectic Euler method with a step size of 0.01 to guarantee accuracy. However, in the data used to train the neural networks, a step size of $h = 0.1$ is used. All trajectories are calculated for one entire period T until they start to repeat themselves. During training, only the first quarter of the trajectory is used, which creates two areas of the phase space without training data.

An example of one trajectory predicted by the multilayer perceptron and both PGHNNs can be seen in Figure 3. The trajectories begin at the leftmost point of the red line in the phase space plots and start by moving up and to the right. It can be clearly seen that, in the beginning, all neural networks' predictions follow the data well. However, the predictions of the multilayer perceptron deteriorate quickly after they move outside of the training data. There, the predictions get stuck and no longer change. The predictions of the PGHNNs, by contrast, follow the data very closely and are perfect for the PGHNN with two hidden layers inside the MLPs.

A similar picture presents itself if one studies the mean error of the 10 neural networks over all trajectories in the test data. In Figure 4, it becomes clear that even within the training data, the two PGHNNs have errors of about one order of magnitude less compared to the multilayer perceptron. Moreover, the variation of the errors of the PGHNNs is rather small. This indicates that even for differently initialized PGHNNs, the Adam algorithm finds comparable minima, and the PGHNNs are not sensitive to how the weights are initialized. The PGHNNs with two hidden layers per learned Hamiltonian appear to be the best choice for the single pendulum data.

6.2. Double Pendulum

To build a double pendulum, one must add another rod to the mass of the single pendulum. The second rod is again able to rotate freely in two spatial dimensions and has a second mass at its end. The resulting Hamiltonian system has two additional dimensions ($2d = 4$) and also two additional parameters. Again, we use dimensionless values and fix the masses and the gravitational acceleration ($m_1 = m_2 = 1, g = 1$). Even though the system has only two additional

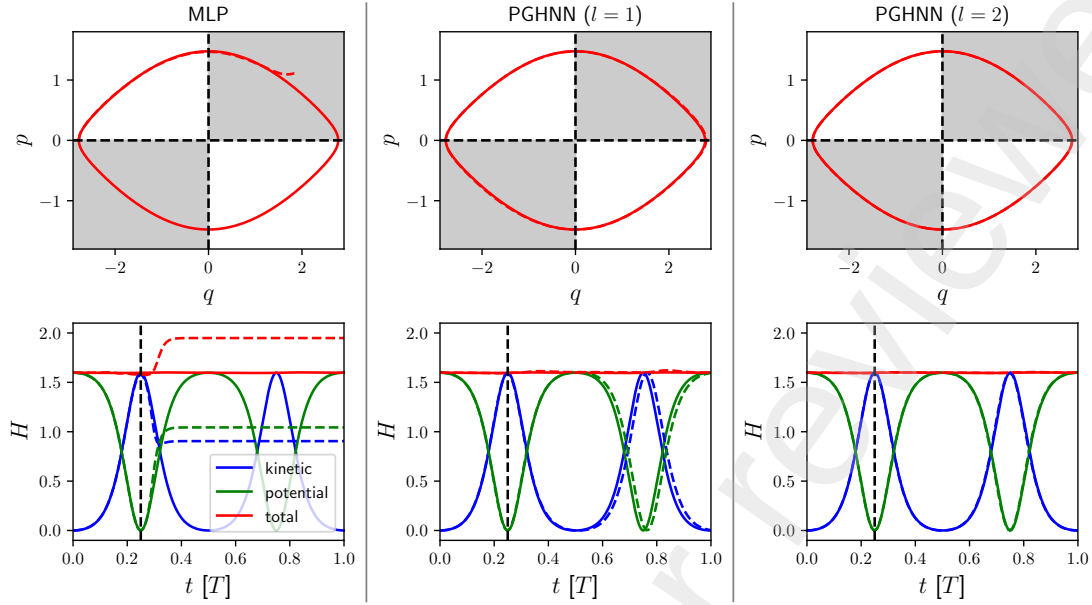


Figure 3: Predictions of an MLP and two PGHNNs for a mathematical pendulum trajectory with an initial angle of $q \approx -0.88\pi$ and the length of the pendulum being $l \approx 0.83$. The upper plots show the phase space while the lower plots show the energy over the entire period. In all plots, the solid lines are the data and the dashed lines are the neural network predictions. In the white areas of the phase space, there is training data but in the gray ones, there is none.

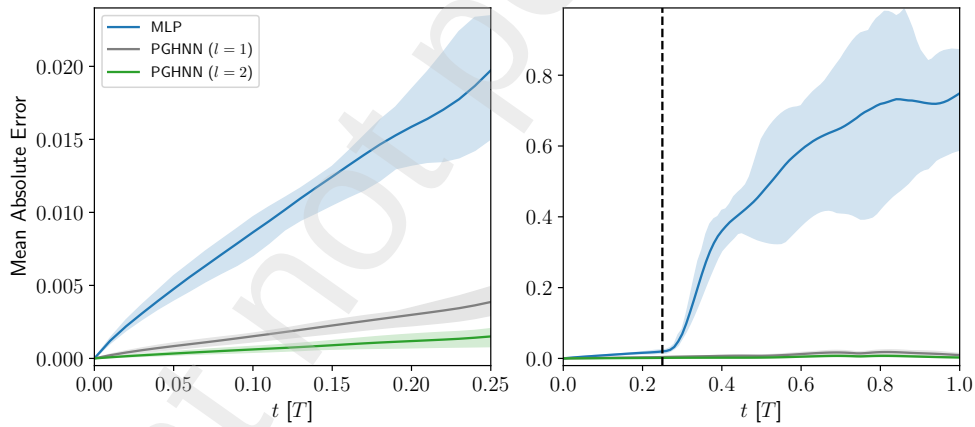


Figure 4: Mean error of the 10 different neural networks over all the trajectories in the test data of the mathematical pendulum (solid lines) and ranges of errors enclosed by the 10% and 90% quantiles (plumes around solid lines). The left plot is inside the training time range, the right plot shows the errors over the entire time range considered.

dimensions, it is considerably more complex. The double pendulum is well known for its chaotic nature; furthermore, the Hamiltonian of this system is more complicated and no longer separable:

$$H(\mathbf{p}, \mathbf{q}) = \frac{l_2^2 p_1^2 + 2l_1^2 p_2^2 - 2l_1 l_2 p_1 p_2 \cos(q_1 - q_2)}{2l_1^2 l_2^2 (1 + \sin^2(q_1 - q_2))} + 2l_1(1 - \cos(q_1)) + l_2(1 - \cos(q_2)) + l_2. \quad (16)$$

The dataset for the double pendulum is built similarly to that for the single pendulum. Overall, 2000 trajectories are calculated using the symplectic Euler method. The step size for the numerical integrator is not fixed; it is reduced until the calculations converge. This is done to ensure that the data is close to the ground truth. For the training data,

again a constant step size of $h = 0.1$ is used. In contrast to other works (e.g. [4]), we use a symplectic integrator instead of a higher-order integrator to ensure that the data on which the neural networks are trained comes from a symplectic map. Again, all momenta are zero initially, and the angles are random ($q_1, q_2 \in [-0.5\pi, 0.5\pi]$). For the double pendulum, both lengths are varied at random ($l_1, l_2 \in [0.5, 1.5]$), and therefore the parameter input μ to the neural networks now has a dimension of 2. All trajectories are calculated until a final time of $t_{\text{end}} = 10$ is reached, but for the training, the trajectories are reduced to include only the first half.

For the double pendulum, Figure 5 shows that there is almost no difference between the region of the trajectories that is within the training data and the region that is outside. This is because there is no periodicity in the trajectories

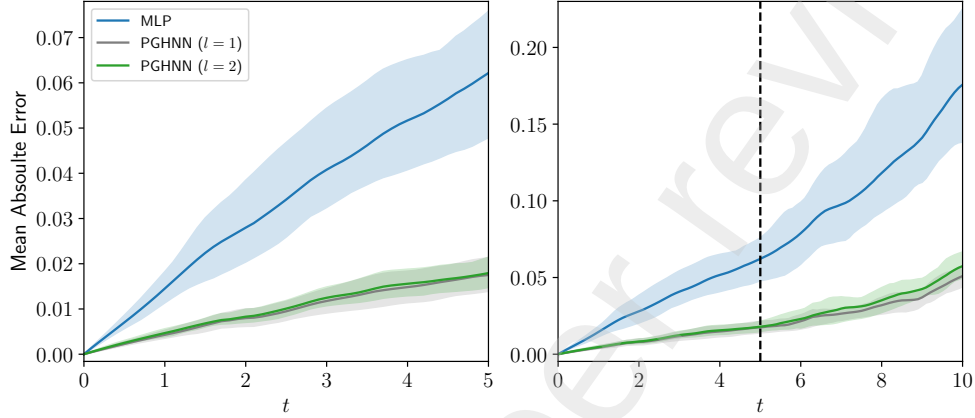


Figure 5: Mean error of the 10 different neural networks over all the trajectories in the test data of the double pendulum (solid lines) and ranges of errors enclosed by the 10% and 90% quantiles (plumes around solid lines). The left plot is inside the training time range, the right plot shows the errors over the entire time range considered.

of the double pendulum, and thus the cut of the training data after $t = 5$ does not create a region of the phase space without data. Again, the PGHNNs clearly outperform the multilayer perceptrons. Even the best multilayer perceptron has an error that is about twice as high as that of the worst PGHNN. For the double pendulum, the two different PGHNN topologies have similar errors, and it cannot be determined which topology is the best.

Apart from this, the benefit of including the symplectic structure within the neural networks becomes evident in the energy plots of Figure 6. While the total energy in the system predicted by the multilayer perceptron grows quickly and then falls over large portions of the predicted trajectory, the total energies of the structure-preserving neural networks' predictions only fluctuate around the correct energy. The amplitudes of these fluctuations are larger than those of the symplectic integrator, but not significantly larger.

6.3. Double Pendulum with Irregular Step Size

In a multilayer perceptron, a dependency on a varying step size can be easily implemented. Like a dependency on a parameter, the step size is added through an additional input neuron without any other changes to the neural network. Therefore, we can compare three different neural network architectures in this numerical experiment: multilayer perceptrons, SympNets, and Parametric Generalized Hamiltonian Neural Networks.

For this experiment, data similar to the double pendulum data in the paper introducing the Generalized Hamiltonian Neural Networks [6] is used. Again, the data consists of 2000 trajectories with the same initial conditions as in the previous experiment ($p_1 = p_2 = 0, q_1, q_2 \in [-0.5\pi, 0.5\pi]$). All parameters in the double pendulum system are fixed ($l_1 = l_2 = 1, m_1 = m_2 = 1, g = 1$). The only difference to the data of [6] is a varying step size. The step size h for every feature/label pair is drawn uniformly from $[0.05, 0.15]$, whereas it was previously fixed to 0.1.

We choose this double pendulum data as a test case since the SympNets and Generalized Hamiltonian Neural Networks with two hidden layers show almost identical performance on the data with a fixed step size, with the SympNets predicting slightly more accurate trajectories on average. This allows us to separate the influence of how the step size is included in the neural network from the differences in the rest of the neural network architecture. To

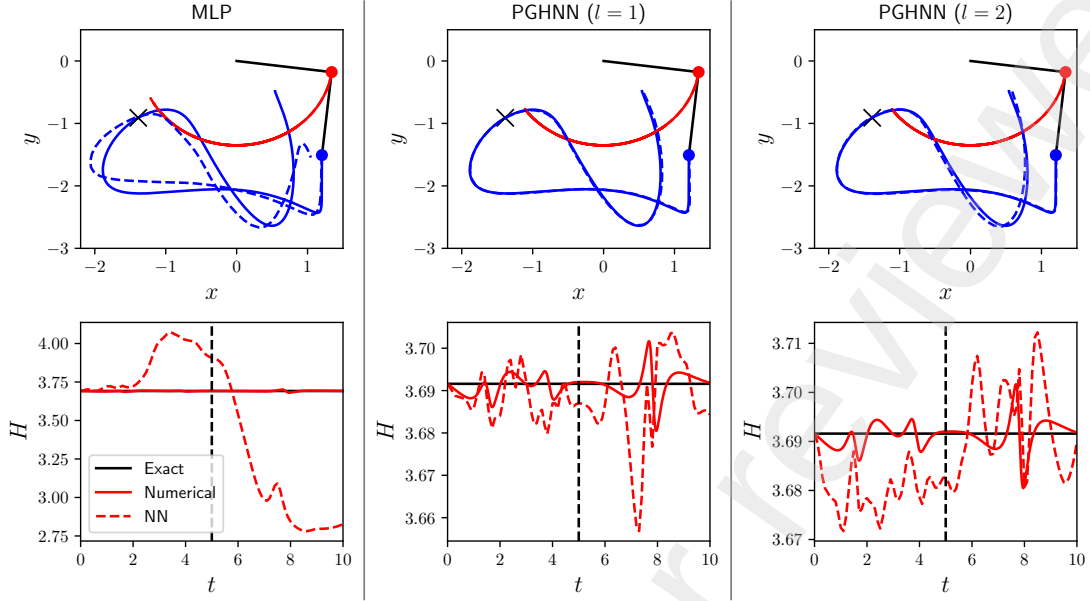


Figure 6: Predictions of an MLP and two PGHNNs for a double pendulum trajectory with initial angles of $q_1 \approx 0.46\pi$, $q_2 \approx -0.03\pi$ and the lengths of the pendulum being $l_1 \approx 1.35$, $l_2 \approx 1.34$. The upper plots show the physical space while the lower ones show the energy evolution. In all plots, the solid lines are the data and the dashed lines are the neural network predictions. The black $\times$ in the upper plots marks the end of the training data.

further separate these two factors, PGHNNs with one and two hidden layers per learned Hamiltonian are trained. A GHNN with one hidden layer per learned Hamiltonian is equivalent to a SympNet. Therefore, the only difference between the PGHNNs with one hidden layer and the SympNets is the way the step size is incorporated.

The results from this numerical experiment are clearly in favor of including the step size in a non-linear way, as Figure 7 shows. Inside the training data, the SympNets predict the trajectories on average less accurately than all three neural network topologies with a direct non-linear influence of the step size. To calculate the error, a step size of $h = 0.1$ is used for the prediction of all trajectories. This is right in the middle of the range of different step sizes that are used for training. The PGHNNs predict the most accurate trajectories and can maintain the same slow and stable accumulation of error both inside and outside of the training data. The multilayer perceptrons outperform the SympNets within the training data. However, outside of the training data, the SympNets benefit from their symplectic structure, and their error rises at a stable rate. The error of the multilayer perceptrons increases quickly as soon as the training time range ends.

This abrupt increase in the error of the multilayer perceptrons can be explained by the energy behavior seen in Figure 8. The multilayer perceptron can predict trajectories with stable energy within the training data. However, the energy outside the training data suddenly decreases. Due to the symplectic structure, the energies of the trajectories predicted by the SympNet and the PGHNN seem to be stable and oscillate around the correct energy. These oscillations have a greater amplitude in the prediction of the SympNet, which explains the higher error of the SympNets. Overall, the trajectory predicted by the SympNet follows the data quite well, but the trajectory predicted by the PGHNN perfectly matches the correct trajectory. Only a hint of the dashed line in the lower left plot of Figure 8, indicating the predictions of the PGHNN, can be seen.

6.4. Validation Loss during Training

Figure 9 shows the evolution of the validation loss during the training of all neural networks trained for this work. Interestingly, the multilayer perceptron for the double pendulum data is the only neural network showing signs of overfitting. The validation loss starts to increase after around 4000 epochs, but the training loss (not plotted here)

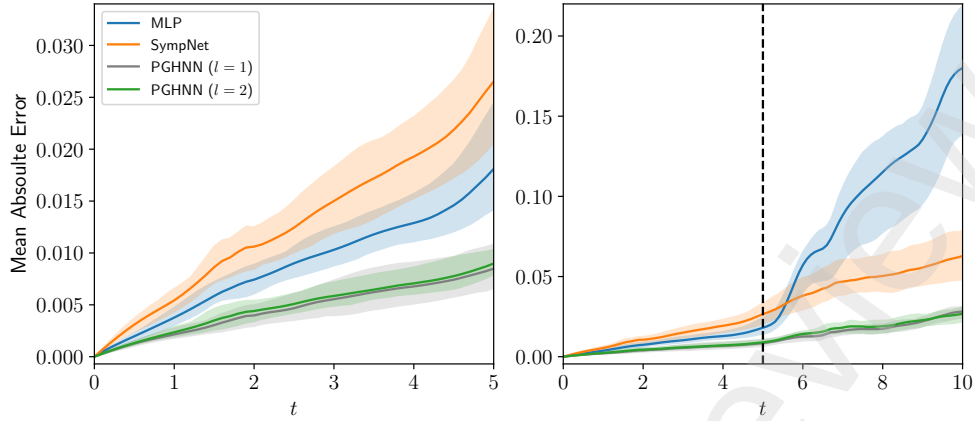


Figure 7: Mean error of the 10 different neural networks over all the trajectories in the test data of the double pendulum with varying step sizes (solid lines) and ranges of errors enclosed by the 10% and 90% quantiles (plumes around solid lines). The left plot is inside the training time range, the right plot shows the errors over the entire time range considered.

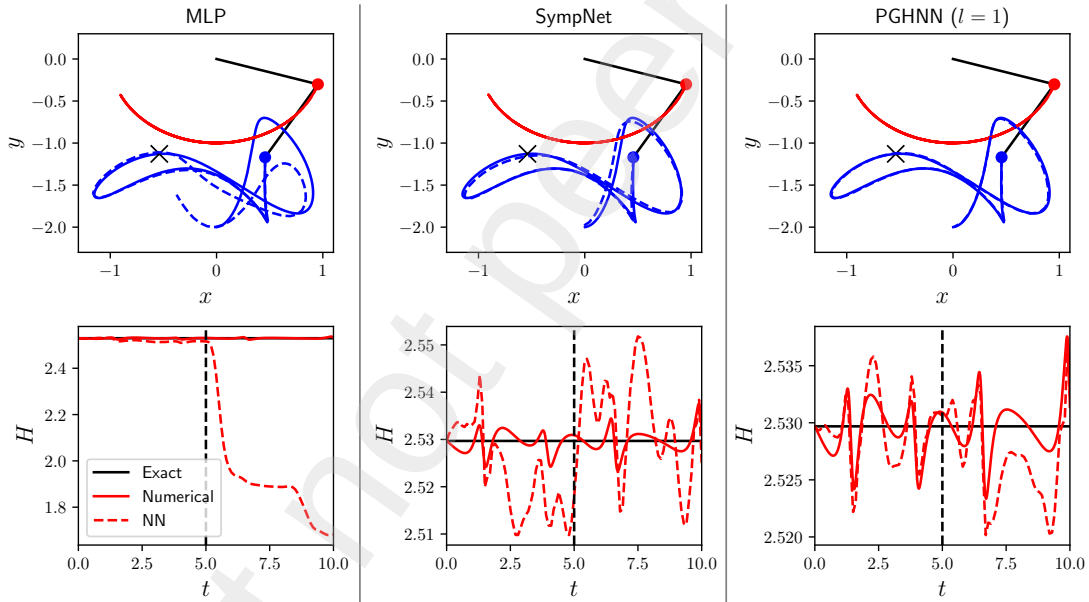


Figure 8: Predictions of an MLP, a SympNet and a PGHNN for a double pendulum trajectory with initial angles $q_1 \approx 0.4\pi$, $q_2 \approx -0.17\pi$. All neural networks use a step size of $h = 0.1$ for the prediction. The upper plots show the physical space while the lower ones show the energy evolution. In all plots, the solid lines are the data and the dashed lines are the neural network predictions. The black $\times$ in the upper plots marks the end of the training data.

continues to decrease. In the numerical experiments in Section 6, we always keep track of the validation loss and, in the end, use the weights and biases that minimize the validation loss, not necessarily the ones after 25000 epochs.

7. Conclusion

In Section 4, we presented a new neural network architecture for Hamiltonian systems. These Parametric Generalized Hamiltonian Neural Networks (PGHNNs) extend the generalized framework of neural networks for Hamiltonian systems (GHNNs) to systems with one or more changing parameters. They preserve the symplectic structure of the

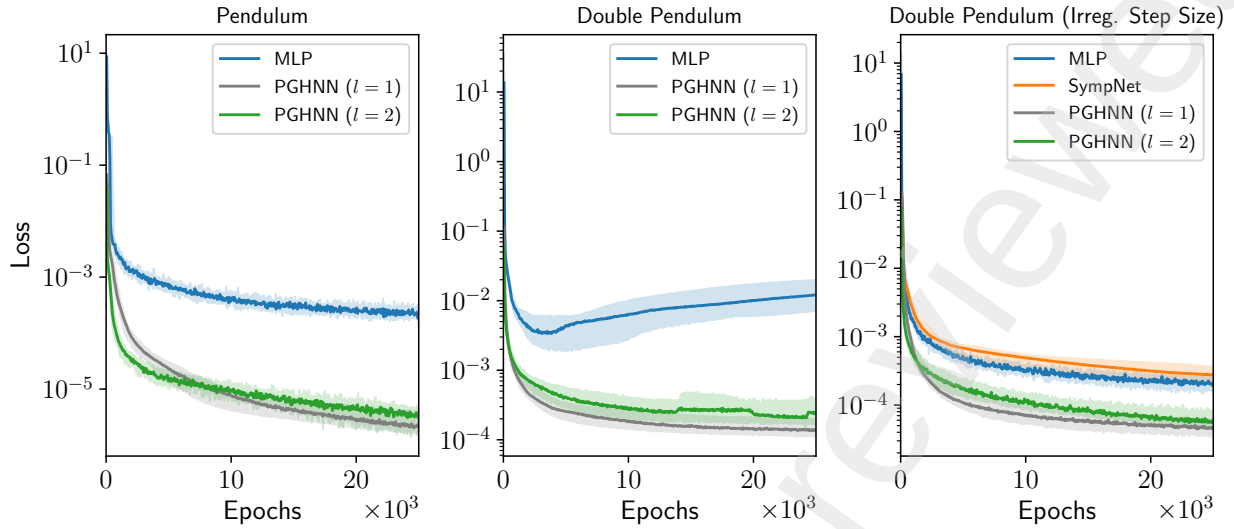


Figure 9: The median validation loss of the 10 neural networks per architecture is plotted using a solid line. The plumes around the solid lines are the 10% and 90% quantiles of the losses.

flow map of Hamiltonian systems and are able to learn a flow map that depends on one or more parameters in addition to the phase space coordinates. The additional parameter input can also be a varying step size. This means a PGHNN can, in contrast to a GHNN, also be trained on data that is recorded with irregular step sizes.

We were able to prove a universal approximation theorem showing that for every parameter-dependent r -smooth symplectic map φ and every $\epsilon > 0$, there exists a PGHNN that can approximate φ uniformly in the C^r -norm with precision ϵ . This theorem highlights the fact that the expressivity of the hypothesis space is not limited by the imposed symplectic structure when PGHNNs are used to approximate the flow map of a parametric Hamiltonian system.

In the numerical experiments, it was shown that the PGHNNs achieve considerably higher accuracy than physics-unaware multilayer perceptrons in predicting the trajectories of the single and double pendulum. For this, they need around one order of magnitude fewer trainable parameters. Furthermore, the prediction speeds are the same for the different neural network architectures, even though the outputs are computed on a GPU and the computations of a multilayer perceptron are better parallelized.

On data with irregular step sizes, the PGHNNs also outperform the structure-preserving SympNets and the multilayer perceptrons, both within the training data and outside of it. This is especially remarkable since, on the same data but with constant step sizes, the SympNets can match or even slightly surpass the accuracy of the GHNNs.

The most important benefit of using the structure-preserving PGHNNs over multilayer perceptrons is the greatly improved generalization in areas of the phase space without training data. This is best illustrated by the results of the single pendulum, where the PGHNN can predict the motion of a pendulum for an entire period even though there is training data in only half of the phase space. The multilayer perceptron, on the other hand, completely fails to predict the trajectory in the region of the phase space without training data.

Acknowledgment

This work was funded by the Dutch Research Council (NWO), in the framework of the program ‘Unraveling Neural Networks with Structure-Preserving Computing’ (file number OCENW.GROOT.2019.044).

Appendix A. Proof of the Universal Approximation Theorem

To prove the universal approximation theorem of our PGHNNs, we first take a look at a similar theorem along with its proof from a paper by Turaev [11]. The universal approximation theorems of the SympNets and HénonNets

are also based on this theorem.

Theorem 2 (Theorem 2 of [11]). *For any $r \in \mathbb{N}^0$, open set $U \subset \mathbb{R}^{2d}$, compact set $W \subset U$, any $\varphi \in \mathcal{SP}^{r+1}(U)$, and any $\epsilon > 0$, there exists a sequence of polynomial Hénon-like maps $f_1, \dots, f_{4N}$ such that*

$$\|\varphi - f_{4N} \circ \dots \circ f_1\|_{C^r(W; \mathbb{R}^{2d})} < \epsilon. \quad (\text{A.1})$$

Remark. *The parameter-dependent case is discussed in a remark after this theorem is stated in [11]. It is noted that if φ is C^{r+1} -smooth with respect to $(\mathbf{p}, \mathbf{q}, \boldsymbol{\mu})$, then the functions f_i are C^r -smooth with respect to $(\mathbf{p}, \mathbf{q}, \boldsymbol{\mu})$ and the composition $f_{4N} \circ \dots \circ f_1$ is ϵ -close to φ , along with all the derivatives, uniformly on any compact set in the $(\mathbf{p}, \mathbf{q}, \boldsymbol{\mu})$ -space. This more general parameter-dependent case is also what is considered in the proof.*

The proof of this theorem relies heavily on a lemma that is also introduced and proven in [11]. So, let us first take a look at this lemma and then continue with the proof of Theorem 2. Before we state this lemma, there is one more detail regarding the notation. The flow map φ of a Hamiltonian system is usually denoted with two indices: the time t and the step size h . So,

$$\varphi_{t,h} : \mathbf{x}(t) \mapsto \mathbf{x}(t+h). \quad (\text{A.2})$$

In autonomous Hamiltonian systems discussed thus far, systems with a Hamiltonian that does not depend on time, the flow map is independent of the time t , and so only the step size is added as an index to the flow map.

Lemma 1 (Lemma 1 of [11]). *Given any $H(\mathbf{p}, \mathbf{q}, \boldsymbol{\mu}, t)$ (polynomial), there exists a function $V(\mathbf{q}, \boldsymbol{\mu}, \tilde{t})$ such that, on any compact domain in the $(\mathbf{p}, \mathbf{q}, \boldsymbol{\mu})$ -space (and any compact interval of values of t), the flow map $\varphi_{t, \frac{1}{m}}$ of the system:*

$$\begin{pmatrix} \dot{\mathbf{p}} \\ \dot{\mathbf{q}} \end{pmatrix} = -J \nabla H(\mathbf{p}, \mathbf{q}, \boldsymbol{\mu}, t), \quad (\text{A.3})$$

is uniformly $O(\frac{1}{m^2})$ -close to the flow map $\tilde{\varphi}_{0,2\pi}$ of the system:

$$\begin{pmatrix} \dot{\mathbf{p}} \\ \dot{\mathbf{q}} \end{pmatrix} = \begin{pmatrix} -\text{diag}(\Omega)^2 \mathbf{q} - \frac{1}{m} \nabla_{\mathbf{q}} V(\mathbf{q}, \boldsymbol{\mu}, \tilde{t}) \\ \mathbf{p} \end{pmatrix}, \quad (\text{A.4})$$

where $\Omega = (\Omega_1, \dots, \Omega_d)^T$, with appropriately chosen integers Ω .

Proof. The proof can be found in [11]. □

With this lemma, we now provide a slight reformulation of the proof of Theorem 2 compared to how it is presented in [11]. In our proof, we change the order in which the different arguments are presented, and some notations are adjusted to make them more consistent with the rest of our work.

Proof of Theorem 2. First we approximate the flow map $\tilde{\varphi}_{0,2\pi}$ of a system of the form (A.4). To do this we split the full 2π flow map into several flow maps $\tilde{\varphi}_{\tilde{t},h}$ with a smaller step size h :

$$\tilde{\varphi}_{0,2\pi} = \tilde{\varphi}_{2\pi-h,h} \circ \dots \circ \tilde{\varphi}_{0,h}. \quad (\text{A.5})$$

Now since

$$\tilde{\varphi}_{\tilde{t},h} \begin{pmatrix} \mathbf{p} \\ \mathbf{q} \end{pmatrix} = \begin{pmatrix} \mathbf{p} - h \nabla_{\mathbf{q}} \tilde{V}(\mathbf{q}, \boldsymbol{\mu}, \tilde{t}) \\ \mathbf{q} + h \mathbf{p} \end{pmatrix} + O(h^2), \quad (\text{A.6})$$

with

$$\tilde{V}(\mathbf{q}, \boldsymbol{\mu}, \tilde{t}) = \mathbf{q}^T \text{diag}(\Omega)^2 \mathbf{q} + \frac{1}{m} V(\mathbf{q}, \boldsymbol{\mu}, \tilde{t}), \quad (\text{A.7})$$

$\tilde{\varphi}_{\tilde{t},h}$ can be approximated uniformly $O(h^2)$ -close by the symplectic map

$$\hat{\varphi}_{\tilde{t},h} \begin{pmatrix} \mathbf{p} \\ \mathbf{q} \end{pmatrix} = \begin{pmatrix} \mathbf{p} - h \nabla_{\mathbf{q}} \tilde{V}(\mathbf{q} + h \mathbf{p}, \boldsymbol{\mu}, \tilde{t}) \\ \mathbf{q} + h \mathbf{p} \end{pmatrix}. \quad (\text{A.8})$$

Then, one can conclude that

$$\|\tilde{\varphi}_{0,2\pi} - \hat{\varphi}_{2\pi-h,h} \circ \cdots \circ \hat{\varphi}_{0,h}\|_{C^r} = O(h). \quad (\text{A.9})$$

Now using Lemma 1 and by using a similar splitting of $\varphi_{0,1}$ as before for $\tilde{\varphi}_{0,2\pi}$:

$$\varphi_{0,1} = \varphi_{1-\frac{1}{m}, \frac{1}{m}} \circ \cdots \circ \varphi_{0, \frac{1}{m}}, \quad (\text{A.10})$$

one can show that

$$\|\varphi_{0,1} - \tilde{\varphi}_{0,2\pi}^m \circ \cdots \circ \tilde{\varphi}_{0,2\pi}^1\|_{C^r} = O\left(\frac{1}{m}\right). \quad (\text{A.11})$$

Here $\tilde{\varphi}_{0,2\pi}^i$ are the flow maps of different Hamiltonian systems, all of the form given in equation (A.4).

Now finally, we use the (according to Turaev [11]) well known fact that any C^{r+1} -smooth symplectic map on any compact domain in (p, q, μ) -space coincides with the flow map $\varphi_{0,1}$ of a non-autonomous Hamiltonian system, so a system of the form presented in Equation (A.3).

To conclude, we can approximate these $\varphi_{0,1}$ flows uniformly by $\tilde{\varphi}_{0,2\pi}$ flows of different systems (A.4). These $\tilde{\varphi}_{0,2\pi}$ flows on the other hand can be approximated uniformly by symplectic maps of the form (A.8). Combined, this means we are able to approximate any C^{r+1} -smooth symplectic map on any compact domain by concatenations of symplectic maps of the form (A.8).

To show that one can not only approximate any C^{r+1} -smooth symplectic map by concatenations of symplectic maps of the form (A.8), but also by Hénon-like maps, Turaev in [11] now notes that these symplectic maps (A.8) can be realized by the concatenation of four Hénon-like maps. $\square$

In this proof, the Hénon-like maps are introduced only in the last paragraph. Since we want to approximate symplectic maps with PGHNNs and not with Hénon-like maps, this is also the only part that we need to adjust to prove our universal approximation theorem.

To do this, we first take a look at one of the many universal approximation theorems for multilayer perceptrons (MLPs).

Theorem 3. For any $r, m \in \mathbb{N}^+$, compact set $W \subset \mathbb{R}^m$, any $f \in C^r(\mathbb{R}^m)$, and any $\epsilon > 0$, there exists an MLP f_{NN} with one hidden layer such that

$$\|f - f_{\text{NN}}\|_{C^r(W)} < \epsilon \quad (\text{A.12})$$

if the activation function σ in the hidden layer is r -finite.

Proof. The proof can be found in [12]. $\square$

This universal approximation theorem for MLPs is used to prove the following universal approximation theorem for SympNets and will also be used by us to prove the universal approximation theorem for PGHNNs.

Theorem 4 (Universal approximation of G-SympNets). For any $r \in \mathbb{N}^+$, open set $U \subset \mathbb{R}^{2d}$, compact set $W \subset U$, any $\varphi \in \mathcal{SP}^r(U)$, and any $\epsilon > 0$, there exists a G-SympNet φ_{NN} such that

$$\|\varphi - \varphi_{\text{NN}}\|_{C^r(W; \mathbb{R}^{2d})} < \epsilon \quad (\text{A.13})$$

if the activation function σ inside the gradient layers is r -finite.

Proof. The proof can be found in [3]. $\square$

In this proof, Theorem 2 is used, and the key idea is that every Hénon-like map can be approximated arbitrarily closely by a concatenation of four gradient modules. So, a total of 16 gradient layers are needed to approximate any symplectic map of the form (A.8).

Now, the proof of the universal approximation theorem for PGHNNs follows:

Proof of Theorem 1. To prove the universal approximation theorem, one can use the same proof as for Theorem 2 but with an adjusted last paragraph. Instead of realizing every symplectic map $\hat{\varphi}$ of the form (A.8) by a concatenation of four Hénon-like maps, we show that these maps $\hat{\varphi}$ can be approximated arbitrarily close by one symplectic integration step with a separable Hamiltonian that is parameterized by two MLPs. So, exactly one of the concatenated symplectic integrators inside a PGHNN. It is enough to restrict the PGHNNs to the symplectic Euler method and a single hidden layer in all MLPs that learn the separable Hamiltonians.

First, we note that the time $\tilde{t}$ dependency of $\tilde{V}$ in (A.8) is not relevant for us since for every of the concatenated maps $\hat{\varphi}$ the time $\tilde{t}$ has a fixed value. So, we can also treat the time-dependent function $\tilde{V}$ as multiple different time-independent functions like

$$\tilde{V}(\mathbf{q}, \boldsymbol{\mu}, 0) = \tilde{V}_1(\mathbf{q}, \boldsymbol{\mu}), \quad \dots, \quad \tilde{V}(\mathbf{q}, \boldsymbol{\mu}, 2\pi - h) = \tilde{V}_n(\mathbf{q}, \boldsymbol{\mu}). \quad (\text{A.14})$$

It is rather straightforward to see that each map $\hat{\varphi}$ is exactly one step of the symplectic Euler method with step size h and a Hamiltonian H with the following structure:

$$H(\mathbf{p}, \mathbf{q}, \boldsymbol{\mu}) = \underbrace{\mathbf{p}^T \mathbf{p}}_{T(\mathbf{p}, \boldsymbol{\mu})} + \underbrace{\tilde{V}_i(\mathbf{q}, \boldsymbol{\mu})}_{U(\mathbf{q}, \boldsymbol{\mu})}. \quad (\text{A.15})$$

Both the kinetic and potential energy of this Hamiltonian are r -smooth for any $r \in \mathbb{N}^0$. Furthermore, the activation functions of all MLPs inside the integrators of the PGHNN are $(r + 1)$ -finite. Hence, we can use Theorem 3 to guarantee that for every ϵ there exists an MLP T_{NN} and an MLP U_{NN} such that

$$\|H - H_{\text{NN}}\|_{C^{r+1}(W; \mathbb{R}^{2d})} = \|T - T_{\text{NN}} + U - U_{\text{NN}}\|_{C^{r+1}(W; \mathbb{R}^{2d})} < \epsilon, \quad (\text{A.16})$$

where $H_{\text{NN}} = T_{\text{NN}} + U_{\text{NN}}$.

Approximating the Hamiltonian in the C^{r+1} -norm is necessary since its gradients are used in the symplectic Euler method. Together this implies

$$\|\hat{\varphi} - \text{SI}(H_{\text{NN}}, \cdot, \cdot)\|_{C^r(W; \mathbb{R}^{2d})} = \|\nabla H - \nabla H_{\text{NN}}\|_{C^r(W; \mathbb{R}^{2d})} \leq \|H - H_{\text{NN}}\|_{C^{r+1}(W; \mathbb{R}^{2d})} < \epsilon. \quad (\text{A.17})$$

With this adjustment one can use the proof of Theorem 2 to show that: a concatenation of symplectic Euler methods performing one integration step with a separable Hamiltonian parametrized by two MLPs can approximate any $\varphi \in \mathcal{SP}^{r+1}(U)$ on a compact set with arbitrary accuracy as long as the activation functions inside the MLPs are $(r + 1)$ -finite. This concatenation of symplectic Euler methods is a subclass of the PGHNNs. Therefore, any φ can be approximated by a PGHNN with arbitrary accuracy. $\square$

One could also prove this universal approximation theorem by copying the proof of the universal approximation theorem for SympNets with only minor adjustments for the parameter-dependent case. However, that would imply that 8 concatenated symplectic integrators are required to approximate one symplectic map of the form (A.8), when in fact, one integrator is enough.

References

- [1] S. Greydanus, M. Dzamba, J. Yosinski, Hamiltonian neural networks, in: Proceedings of Advances in Neural Information Processing Systems (NeurIPS 2019), Vol. 32, Curran Associates, Inc., 2019.
- [2] Z. Chen, J. Zhang, M. Arjovsky, L. Bottou, Symplectic recurrent neural networks, in: Proceedings of the 8th International Conference on Learning Representations (ICLR 2020), 2020.
- [3] P. Jin, Z. Zhang, A. Zhu, Y. Tang, G. Karniadakis, SympNets: Intrinsic structure-preserving symplectic networks for identifying Hamiltonian systems, *Neural Networks* 132 (2020) 166–179.
- [4] J. Burby, Q. Tang, R. Maulik, Fast neural Poincaré maps for toroidal magnetic fields, *Plasma Physics and Controlled Fusion* 63 (2021) 024001.

- [5] S. Xiong, Y. Tong, X. He, S. Yang, C. Yang, B. Zhu, Nonseparable symplectic neural networks, in: Proceedings of the 9th International Conference on Learning Representations (ICLR 2021), 2021.
- [6] P. Horn, V. Saz Ulibarrena, B. Koren, S. Portegies Zwart, A generalized framework of neural networks for Hamiltonian systems, *Journal of Computational Physics* 521 (2025) 113536.
- [7] M. Raissi, P. Perdikaris, G. E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *Journal of Computational Physics* 378 (2019) 686–707.
- [8] E. Hairer, C. Lubich, G. Wanner, *Geometric Numerical Integration*, Springer, 2006.
- [9] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, PyTorch: An imperative style, high-performance deep learning library, in: *Advances in Neural Information Processing Systems*, Vol. 32, Curran Associates, Inc., 2019, pp. 8024–8035.
- [10] D. Kingma, J. Ba, Adam: A method for stochastic optimization, in: *Proceedings of the 3rd International Conference on Learning Representations (ICLR 2015)*, 2015.
- [11] D. Turaev, Polynomial approximations of symplectic dynamics and richness of chaos in non-hyperbolic area-preserving maps, *Nonlinearity* 16 (1) (2002) 123.
- [12] K. Hornik, M. Stinchcombe, H. White, Universal approximation of an unknown mapping and its derivatives using multilayer feedforward networks, *Neural Networks* 3 (5) (1990) 551–560.